

Coding of Voting Smart Contract-Solidity Language

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VotingSystem {

    // Structure for each candidate
    struct Candidate {
        string name;
        uint voteCount;
    }

    // Addresses that have already voted
    mapping(address => bool) public hasVoted;

    //List of candidates
    Candidate[] public candidates;

    // Contract owner
    address public owner;

    // Runs only once at deployment
    constructor(string[] memory candidateNames) {
        owner = msg.sender;

        // Add initial candidates
        for (uint i = 0; i < candidateNames.length; i++) {
            candidates.push(Candidate({
                name: candidateNames[i],
                voteCount: 0
            }));
        }
    }

    // Only owner can add candidates
    function addCandidate(string memory name) public {
        require(msg.sender == owner, "Only owner can add candidates");

        candidates.push(Candidate({
            name: name,
            voteCount: 0
        }));
    }

    // voting
```

Coding of Voting Smart Contract-Solidity Language

```
function vote(uint candidateIndex) public {
    require(!hasVoted[msg.sender], "You already voted");
    require(candidateIndex < candidates.length, "Invalid candidate");

    hasVoted[msg.sender] = true;
    candidates[candidateIndex].voteCount++;
}

// Get total number of candidates
function getCandidatesCount() public view returns (uint) {
    return candidates.length;
}

// Get a candidate's result
function getCandidate(uint index)
    public
    view
    returns (string memory, uint)
{
    Candidate memory c = candidates[index];
    return (c.name, c.voteCount);
}
```